

Reviewer Pack — Minimal Number of Geometric Realizations Bounds Circuit Depth...

Entry e4870a6b62e7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 16:54:30 UTC

Minimal Number of Geometric Realizations Bounds Circuit Depth for XOR Circuits

Entry ID: e4870a6b62e7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 16:54:30 UTC

1. Conjecture

Field A (mathematical branch): Geometric Realization Theory × Boolean Function Complexity: Circuit Complexity **Field B** (complexity object): XOR Circuit Complexity

Statement:

For every boolean function f with n inputs and depth d , the minimal number of geometric realizations required to represent f is $O(d^{(2/3)^n}(1/3))$. Specifically, there exists a constant C such that for all $n \leq 40$, the depth of the smallest geometric realization circuit for f is at most $C * (d^{(2/3)^n}(1/3))$.

Rationale (proposer's reasoning):

Geometric Realization Theory has been used to construct objects from abstract algebraic structures. By applying this theory to boolean functions, we may expose a deeper connection between the geometric structure and computational complexity, potentially leading to new algorithms for circuit depth approximation.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cb431a185689397b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if the Spearman rank correlation coefficient (ρ) between the minimal number of geometric realizations and the circuit depth for all tested boolean functions is greater than or equal to 0.5, using at least 100 seeds. The conjecture is falsified if $\rho < 0.3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -geometric realization theory boolean function complexity XOR circuit complexity - circuit depth bounds geometric realizations XOR circuits $O(d^{2/3}n^{1/3})$ - min number geometric realizations bound XOR circuit depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def circuit_depth(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("Invalid function length")
        depth = 0
        while any(x > 1 for x in f):
            new_f = []
            for i in range(0, len(f), 2):
                new_f.append(int(f[i] != f[i + 1]))
            f = new_f
            depth += 1
        return depth

    def geometric_realizations(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("Invalid function length")
        realizations = []
        for i in range(2**n):
            realization = [f[i]]
            while len(realization) < n:
                next_val = (realization[-1] + 1) % 2
                if next_val not in realization:
                    realization.append(next_val)
```

```

        realizations.append(realization)
    return len(set(tuple(r) for r in realizations))

results = []
for _ in range(100):
    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    depth = circuit_depth(f)
    realizations = geometric_realizations(f)
    results.append((realizations, depth))

if not results:
    return {
        "metric_name": "Spearman rank correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

realizations = [r for r, _ in results]
depths = [d for _, d in results]

def spearman_rank_correlation(realizations, depths):
    n = len(realizations)
    rank_realizations = sorted(range(n), key=lambda i: realizations[i])
    rank_depths = sorted(range(n), key=lambda i: depths[i])
    rho_numerator = sum((rank_realizations[i] - rank_depths[i]) ** 2 for i in range(n))
    rho_denominator = n * (n**2 - 1) / 6
    return 1 - 6 * rho_numerator / rho_denominator

rho = spearman_rank_correlation(realizations, depths)

return {
    "metric_name": "Spearman rank correlation",
    "metric_value": rho,
    "instances_tested": len(results),
    "n_max": max(n for _, n in results),
    "conjecture_holds": rho >= 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(30, 100))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE no_results")
        exit(0)

```

```

mean_rho = sum(r["metric_value"] for r in results) / len(results)
std_rho = math.sqrt(sum((r["metric_value"] - mean_rho) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"rho < 0.5\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the Spearman rank correlation coefficient (ρ) could not be calculated to determine if the conjecture is supported or falsified.
| next: Run the test again with increased time limits and ensure it completes successfully to calculate the Spearman rank correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14966
2	propose	ollama_remote	glm4:latest	0	0	13936
3	propose	ollama_remote	glm4:latest	0	0	11961
4	preregistration	ollama_remote	glm4:latest	0	0	12435
5	novelty	ollama_remote	glm4:latest	0	0	8839
6	novelty	ollama_remote	glm4:latest	0	0	9045
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21694
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18750
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9123
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15330
11	judge	ollama_remote	glm4:latest	0	0	42703

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 178782 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/e4870a6b62e7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e4870a6b62e7.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e4870a6b62e7.tar.gz (if generated)